РЕШЕН ИЗПИТ 4 — С ПЪЛНИ ОБЯСНЕНИЯ

Базови Програмни Езици — C (plain C)

Изпит 4: Клиника за домашни любимци | 100 точки

Теми: struct + createPatient() връща стойност | търсене по float цена | четене от бинарен файл с >= условие

Специфика: createPatient() ВРЪЩА Patient по стойност — не указател!

Задача 3 носи 40 точки — бинарното четене е приоритет.

СЕКЦИЯ 0 — ПРОЧИТ И АНАЛИЗ

КОД / ИЗРАЗ	КАКВО ОЗНАЧАВА ЗА НАС
`Собственик (статичен низ, 30 символа)`	char ownerName[31]; — 30 + 1 = 31
`Чип "A12345B" (фиксирана дължина)`	char chip[8]; — A12345B = 7 символа + \0 = 8
`Цена за преглед (реално, 0.00)`	float price; — при printf: %.2f
`Функцията не приема параметри`	Patient createPatient() — без аргументи!
`Функцията ВРЪЩА създадения елемент`	return p; — по стойност, не указател
`разделени със ',' (запетая)`	animalsText.txt с fprintf(f, "%s,%.2f\n", ...)
`прочита от animalsBin.bin`	fopen("animalsBin.bin", "rb") + fread в while
`равна или по-голяма от параметъра`	price >= minPrice (включително равна!)

СЕКЦИЯ 1 — ПЪЛЕН КОД (gcc -Wall -std=c99)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char  ownerName[31]; /* до 30 символа + '\0' */
    char  chip[8];       /* A12345B = 7 символа + '\0' */
    float price;         /* цена за преглед */
} Patient;

/* — ЗАДАЧА 1: createPatient() — връща по СТОЙНОСТ — */
Patient createPatient() {
    Patient p;  /* локална структура на стека */
    printf("Собственик: "); scanf("%30s", p.ownerName);
    printf("Чип:        "); scanf("%7s",  p.chip);
    printf("Цена:       "); scanf("%f",  &p.price);

    /* Запис в текстов файл с разделител ',' */
    FILE *f = fopen("animalsText.txt", "a");
    if (f) {
        fprintf(f, "%s,%.2f\n", p.ownerName, p.price);
        fclose(f);
    }
    return p;  /* копие на структурата се връща на caller-а */
}

/* — ЗАДАЧА 2: findByPrice — */
void findByPrice(Patient *arr, int count, float price) {
    int found = 0;
    for (int i = 0; i < count; i++) {
        if (arr[i].price == price) {  /* float сравнение — ok за прости цени */
            printf("%s - %s\n", arr[i].ownerName, arr[i].chip);
            found = 1;
            /* НЕ break — може да има повече съвпадения */
        }
    }
    if (!found) printf("Не е намерен запис за цена %.2f.\n", price);
}

/* — ЗАДАЧА 3: четене от бинарен файл — */
void printAbovePrice(const char *fn, float minPrice) {
    FILE *f = fopen(fn, "rb");
    if (!f) { printf("Грешка при %s!\n", fn); return; }

    Patient p;
    while (fread(&p, sizeof(Patient), 1, f) == 1) {
        if (p.price >= minPrice) {  /* >= включва равна! */
            printf("Bin OwnerName: %s\n", p.ownerName);
            printf("Bin Chip: %s\n",      p.chip);
            printf("Bin Price: %.2f\n",  p.price);
        }
    }
    fclose(f);
}

int main() {
    /* Масивът се управлява в main — createPatient само създава */
    Patient *arr = NULL;
```

```
    int count = 0;

    /* Добавяме пациенти ръчно с realloc */
    for (int i = 0; i < 3; i++) {
        count++;
        arr = (Patient*)realloc(arr, count * sizeof(Patient));
        arr[count - 1] = createPatient();  /* присвояване на struct */
    }

    findByPrice(arr, count, 45.50f);
    printAbovePrice("animalsBin.bin", 40.0f);

    free(arr);
    return 0;
}
```

СЕКЦИЯ 2 — КЛЮЧОВИ ОБЯСНЕНИЯ

createPatient() — връщане по стойност

Разлика от addMember() на Изпит 5: тук функцията НЕ управлява масива. Тя просто създава един Patient и го връща. Caller-ът (main) отговаря за realloc и добавянето.

```
/* Два различни подхода — двата са правилни */

/* Подход А: addMember стил (Изпит 5) — функцията управлява масива */
Member* addMember(Member *arr, int *count) {  /* realloc вътре */ }
arr = addMember(arr, &count);  /* caller само получава новия указател */

/* Подход Б: createPatient стил (Изпит 4) — функцията само създава */
Patient createPatient() { return p; }  /* без параметри, без realloc */
arr = realloc(arr, count * sizeof(Patient));  /* caller управлява */
arr[count-1] = createPatient();  /* присвояване на struct */
```

Задача 2: findByPrice() — float сравнение

Сравнението на float с == е рисковано (floating-point precision), но за прости цени с 2 знака след точката (45.50, 58.30) работи на практика в изпитна среда.

> **ВАЖНО:** На изпита float == float е приемливо. В production код се ползва: if (fabsf(arr[i].price - price) < 0.001f)

КОД	ОБЯСНЕНИЕ
`if (arr[i].price == price)`	Float сравнение — работи за прости цени на изпита.
`/* НЕ break */`	Може да има ПОВЕЧЕ пациенти с едната цена. Не спираме след първия.
`if (!found) printf(...)`	Принтираме само ако 0 съвпадения — след края на цикъла.

Задача 3: >= minPrice — включителна долна граница

```
[NE]  if (p.price > minPrice)   // изключва РАВНАТА цена — грешно за Изпит 4!
[OK]  if (p.price >= minPrice)  // равна ИЛИ по-голяма — условието казва 'равна
или по-голяма'
```

Грешки характерни за Изпит 4

```
[NE]  Patient* createPatient()  // задачата иска Patient (стойност), не
Patient*!
```

```
[OK]  Patient createPatient()   // return p; — копие на структурата
```

```
[NE]  fopen("animalsBin.bin", "r")  // "r" е за текстов — данните се четат
грешно!
[OK]  fopen("animalsBin.bin", "rb") // "rb" задължително за бинарни
```

```
[NE]  fprintf(f, "%s;%.2f\n", ...);  // разделителят е ',', не ';'!
[OK]  fprintf(f, "%s,%.2f\n", ...);  // запетая от условието
```